SVKM'S NMIMS SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING

SVKM'S
NMIMS
Deemed-to-be UNIVERSITY

TCS ASSIGNMENT – 1

Name – Nandini Namdeo

Sap ID – 70022400503

B.Tech CE (B)

NANDINI N (1)

ASSIGNMENT-1

NAME:- Nandini Namdeo

CLASS :- Btech CE -'B2'

SAP ID:- 70022400503

ROLL NO:- F088

Ques 1:- What do you understand by the following terms ? Give suitable example.

a) Alphabet :- A finite non-empty set of symbols.

Example:- Alphabet : $\Sigma = \{0, 1\}$

$\Sigma = \{*, \$\}$.

b) String :- A sequence of characters from a given alphabets strings are denoted by lowercase letters.

Example :- If an alphabet is $\Sigma = \{0, 1\}$

So the string will be "010", "1101", "0111".

c) Language :- A language is simply a set of strings formed from some alphabet.

Example:- $\Sigma = \{0, 1\}$

$\Sigma^* = \{\epsilon, 0, 1, 00, 11, 10, 01, \ldots\}$

Language for the string $\Rightarrow$ L = { all strings with odd no of 0's }

$\Sigma = \{01, 0001, 00011, \ldots\}$

NANDINI N

FORMAL LANGUAGE :- A set of strings formed from an alphabet that satisfies specific rules.

Example :- Language of even-length binary strings

$L = \{ w \in \Sigma^* \mid |w| \text{ is even} \}$.

Que2 :- Define Automata.
Explain following finite automata with their tuples. Also comment on formal languages recognized by finite automata.

Automata :- They are used to recognize the formal language.

It's an abstract machine that checks whether a string belongs to a language or not.

i) Deterministic Finite Automata -

$DFA = \{ Q, \Sigma, \delta, q_0, F \}$.

Tuples and parameters.

Q = Finite set of states

Σ = finite set of non-empty input alphabets.

q_0 = Final state

F = Finite set of non-empty final states.

δ = Transition function

$$\boxed{Q \times \Sigma = Q}$$

Non-Deterministic Finite Automata -

$NFA = \{Q, \Sigma, \delta, F, q_0\}$

Tuples and parameters

Q = Finite set of states

Σ = Finite set of non-empty input alphabets

q_0 = final state

F = Finite set of non-empty final states.

δ = Transition function

$$\boxed{Q \times \Sigma = 2^Q}$$

- Finite automata recognize exactly the class of regular languages.
- Regular languages are languages that can be described by -
  - Regular Expression
  - DFA
  - NFA
  - Regular Grammar

ues 3 :- What are the applications of the following in Compiler Constructions?

- Regular Expression -

Regular Expressions are used in lexical analysis, and finite automata help implement the lexer for token recognition.

NANDINI N.

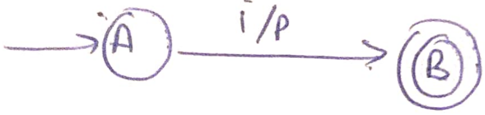
Ques 5 :- Explain and prove various closure properties of Regular Language /Set.

1) Complement
2) Union
3) Concatenation
4) Intersection
5) Reversal
6) Kleen-closure

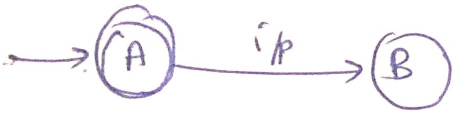
i) Complement -

$L_1 \rightarrow$ is a regular language

~~$L_2 \rightarrow$ is a regular language~~

$M_1 =$ 

$M_1 = \{Q_1, \Sigma_1, \delta_1, q_{01}, F_1\}$



$\overline{M_1} = \{Q_2, \Sigma_2, \delta_2, q_{02}, F_2\}$

$Q_2 = Q_2$

$q_{02} \Rightarrow q_{01}$

$\Sigma_2 = \Sigma_1$

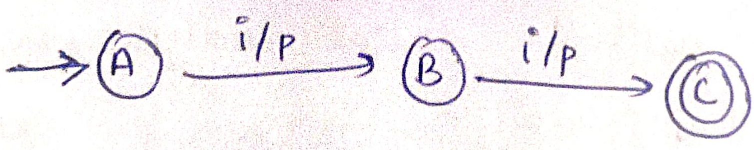
$F_2 = \overline{F_1} \rightarrow (Q - F_1)$

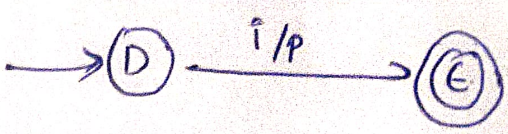
$\delta_2 = \delta_1$

2) Union -

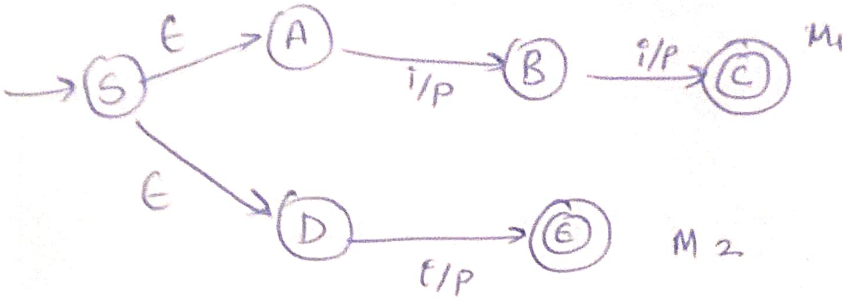
$L_1 \rightarrow$ is a regular language

$L_2 \rightarrow$ is a regular language

M_1 

M_2 

$M_1 = \{Q_1, \Sigma_1, \delta_1, q_0, F_1\}$ $M_2 = \{Q_2, \Sigma_2, \delta_{02}, q_{02}, F_2\}$



$M(L_1 \cup L_2) = \{Q, \Sigma, \delta, q_0, F\}$.

$Q = Q_1 \cup Q_2 \cup S$

$q_0 = S$ → new state

$\Sigma = \Sigma_1 \cup \Sigma_2$

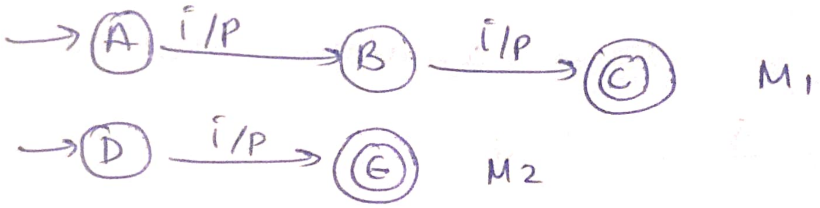
$F = F_1 \cup F_2$

$\delta = \delta_1 \cup \delta_2 \cup \delta(S, \epsilon) = \{q_{01}, q_{02}\}$

3) <u>concatenate</u> —

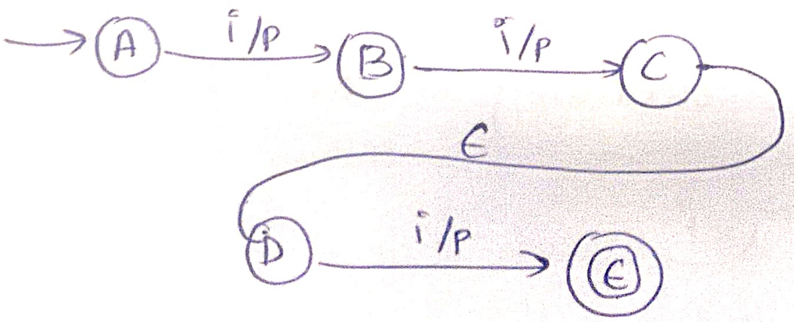
all valid strings of L_2 would be suffix to all valid strings of L_1.

~~$L_1 = \{aa$~~



$M_1 = \{Q_1, \Sigma_1, q_{01}, F_1, \delta_1\}$

$M_2 = \{Q_2, \Sigma_2, q_{02}, F_2, \delta_2\}$



$M(L_1 \cup L_2) = \{Q, \Sigma, q, F, \delta\}$

NANDINI N

$Q = Q_1 \cup Q_2$ $F = F_2$ $\Sigma = \Sigma_1 \cup \Sigma_2$ $\delta = \delta_1 \cup \delta_2 \cup (\xi_1$

$= q_{02}$

$q_0 = q_{01}$

4) Intersection -

$L_1 \cap L_2 = \overline{L_1} \cup \overline{L_2}$ (de- Morgan's law)

L_1 and L_2 are regular language.

$L_1 \rightarrow$ represented by DFA - A with states $\{q_0, q_1, q_2 \ldots\}$

L_2 represented by DFA - B with states $\{p_0, p_1, p_2 \ldots\}$

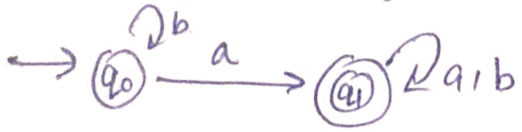
Then, there exists L_3 represented by C.

$L(C) = L(A) \cap L(B)$.

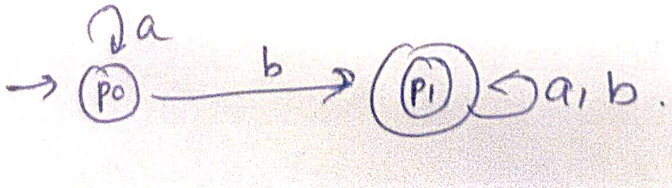
The states of C are pair of (q and p) where q is the state of A and p is a state of B..

The states in C are cross product of A and B.

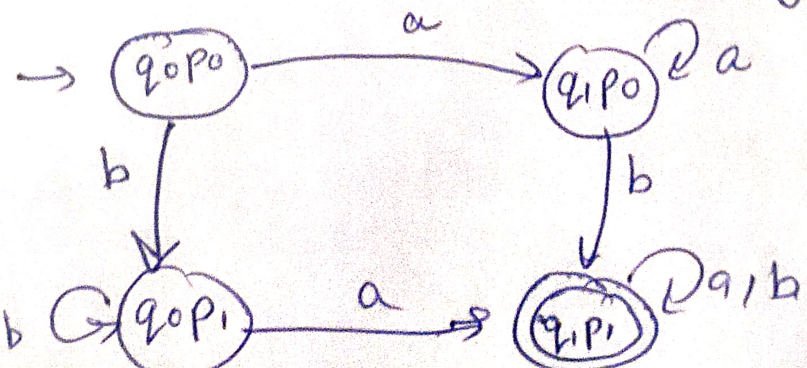
$L_1 \rightarrow$ set of all strings containing 'a' over $\{a, b\}$.



$L_2 \rightarrow$ set of all string containing 'b' over $\{a, b\}$



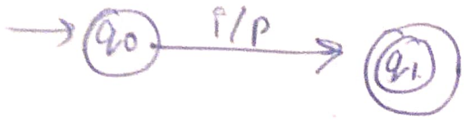
$L_3 \rightarrow L_1 \cap L_2$ [strings containing 'a' and 'b'].



NANDINI (N)

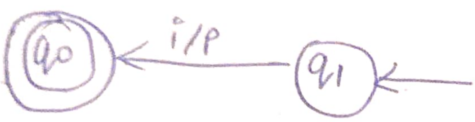
5(a) Reversal –

$L_1 \to$ is a regular language and DFA, exists



$L_{1R} \to$ is reverse of L_1 (Reverse of all strings on L_1).

and new DFA ~~not~~ exists –

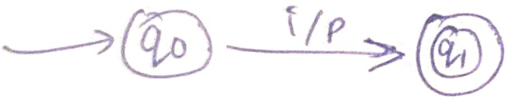


Reversal result – can be NFA / DFA.

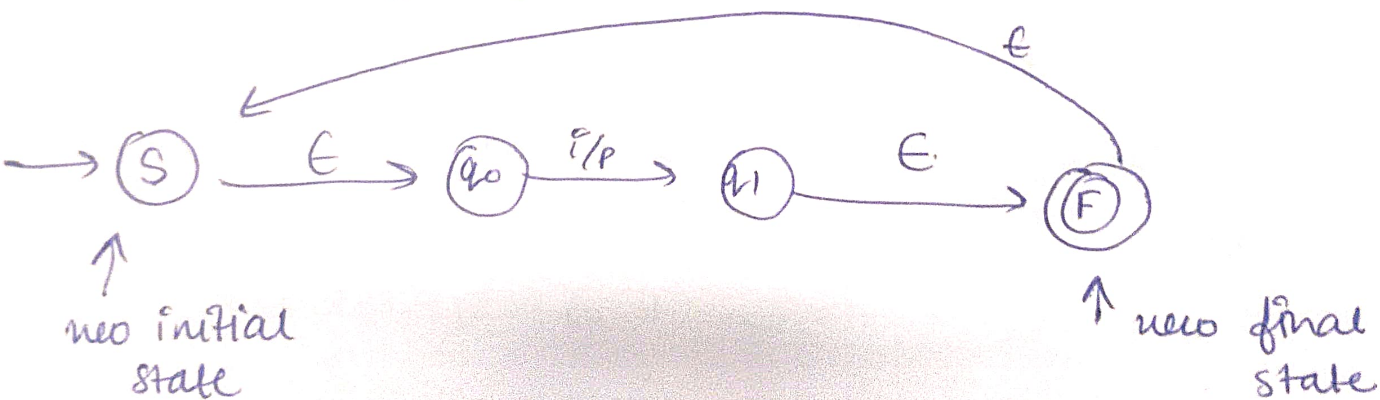
If multiple final state the introduce new final state and final state.

) Kleen- closure –

$L_1 \to L_1$ is regular, DFA exists

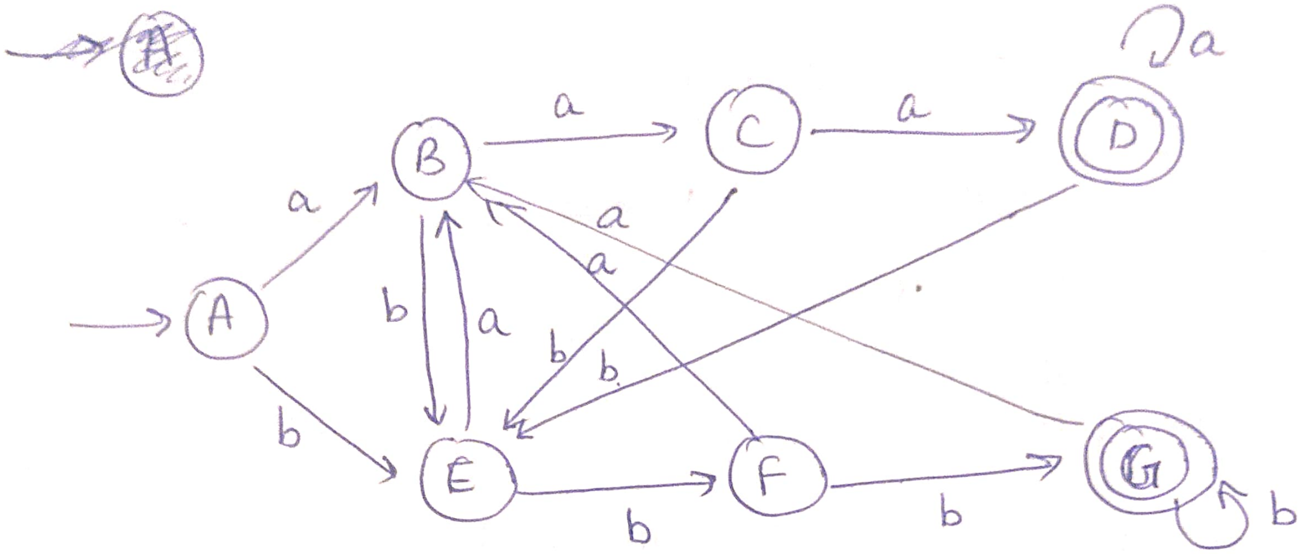


$L_3 \to L_1^*$ is regular, new DFA exists



NANDINI N

Ques → Define DFA to accept all strings ending "aaa" or "b



Tuples of DFA →

DFA = {Q, q_0, δ, F, Σ}

Q = {A, B, C, D, E, F, G}

q_0 → A

F = {D, G}

Σ = {a, b}

δ = (A, a) = B

(A, b) = E

(B, a) = C

(B, b) = E

(C, a) = D

(C, b) = E

(D, a) = D

(D, b) = E

(E, a) = B

(E, b) = F

(F, a) = B

(F, b) = G

(G, a) = B

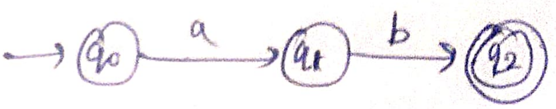
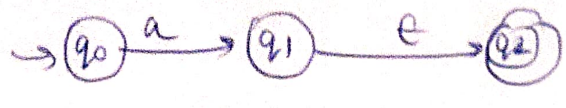
(G, b) = G

Que 7. Differentiate between the following—

a) DFA and NFA.

DFA	NFA
DFA stands for deterministic finite automata.	• NFA stands for non-deterministic automata.
Every input strings leads to the unique state of finite automata.	• For the same input there can be more than one next states.
Tuples of DFA. DFA - $\{Q, q_0, F, \delta, \Sigma\}$ $\delta = Q \times \Sigma = Q$.	• Tuples of NFA $NFA = \{Q, q_0, F, \delta, \Sigma\}$ $\delta = Q \times \Sigma = 2^Q$
There is a compulsion that from each state there should be transition of each input alphabet.	• There is no such compulsion.. Any number of transitions can occur.

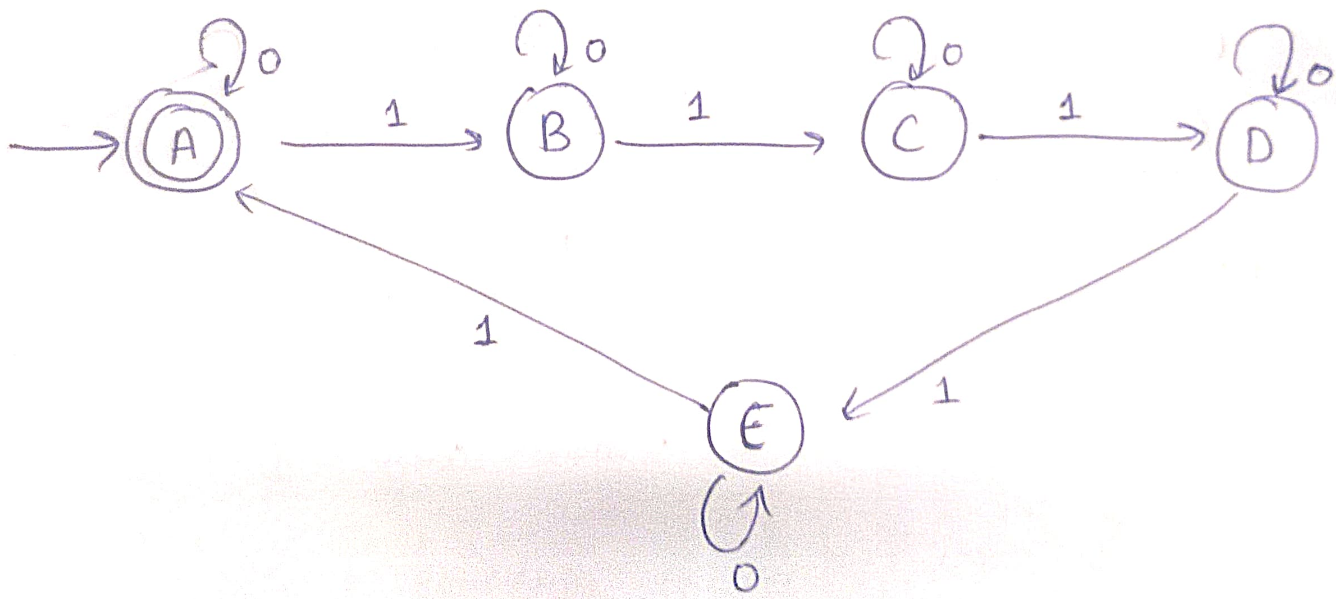
b) NFA and Ɛ-NFA

NFA	Ɛ- NFA
Transitions only on input symbols.	• transitions can also occur without consuming any input
Transition function:- $\delta = Q \times \Sigma = 2^Q$.	• Transition function:- $\delta = Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
construction is slightly harder. 	• Construction is easier. 

C) Regular expressions and regular grammar

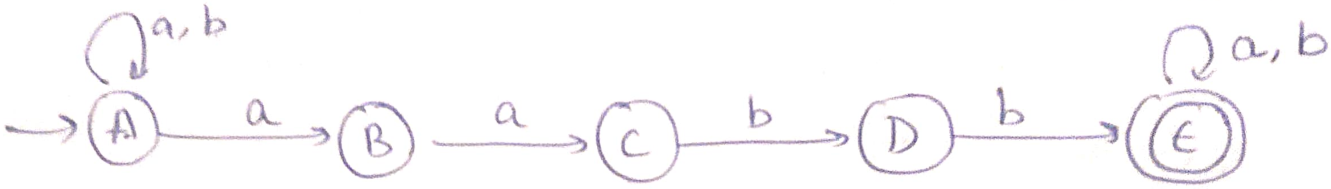
REGULAR EXPRESSION	REGULAR GRAMMAR
◦ An algebraic notation used to describe patterns in strings	◦ A set of production rules used to generate strings.
◦ describes a regular language.	◦ generates a regular language.
◦ uses operators like union (+), concatenation, and kleen star (*).	◦ uses production rules. $A \to aB$ or $A \to a$.
• can be converted to NFA/DFA.	◦ can be converted to finite automata.

Que 8 Design a DFA which accepts string over $\Sigma = \{0,1\}$. in which number of 1's are divisible by 5.



Que 9 Design an NFA with $\Sigma = \{a,b\}$ accepts all string in which 'aa' is followed by 'bb'.

NANDINI N



NFA

ue 10 Convert following NFA to equivalent DFA.

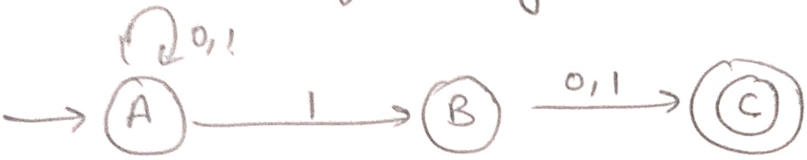
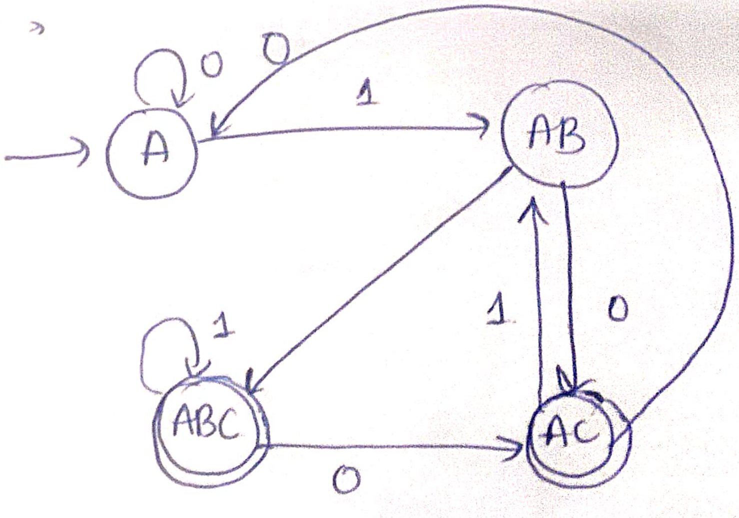


Table of NFA →

Q \ Σ	0	1
→ A	A	A, B
B	C	C
* C	-	-

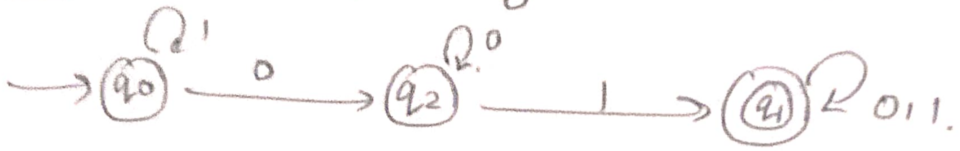
Q \ Σ	0	1
→ A	[A]	[AB]
AB	[AC]	[ABC]
AC	[A]	[AB]
ABC	[AC]	[ABC]

DFA →



NANDINI N

Que 11 Convert following NFA to equivalent DFA



It's already a DFA.

as from each state only one combination of input symbols are transitioning.

Que 12 → construct a regular expression over $\Sigma = \{a, b\}$ for all strings.

i) that start and end with the same symbol.

$a\,(a+b)^*\,a + b\,(a+b)^*\,b + a + b$

ii) that contain atleast one a.

$(a+b)^*\,a\,(a+b)^*$

iii) that do not contain "bb" as a substring.

$(a+ba)^*\,(\varepsilon+b)$

Q13 Minimization of DFA.

State	a	b
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_1	q_2
q_3 (final)	q_3	q_5
q_4	q_6	q_4
q_5 (final)	q_3	q_5
q_6	q_6	q_6

NANDINI N

No state will be eliminated as all the states are reachable from initial state.

0's EQUIVALENT

set of final states = $\{q_3, q_5\}$

set of non-final states = $\{q_0, q_1, q_2, q_4, q_6\}$

1's EQUIVALENT

check for $\{q_0, q_1\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$

$(q_1, a) = q_3$ $(q_1, b) = q_4$

Different set...

check for $\{q_0, q_2\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$

$(q_2, a) = q_1$ $(q_2, b) = q_2$

same set.

check for $\{q_0, q_4\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$

$(q_4, a) = q_6$ $(q_4, b) = q_4$

same set.

check for $\{q_0, q_6\}$.

$(q_0, a) = q_1$ $(q_0, b) = q_2$

$(q_6, a) = q_6$ $(q_6, b) = q_6$

same set.

check for $\{q_3, q_5\}$

$(q_3, a) = q_3$ $(q_3, b) = q_5$

$(q_5, a) = q_3$ $(q_5, b) = q_5$

NANDINI N

1's equivalent –

$\{q_0, q_2, q_4, q_6\}$ $\{q_1\}$ $\{q_3, q_5\}$

2's Equivalent –

Check for $\{q_0, q_2\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$
$(q_2, a) = q_1$ $(q_2, b) = q_2$

same set ✓

Check for $\{q_0, q_4\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$
$(q_4, a) = q_6$ $(q_4, b) = q_4$

not in the same set.

Check for $\{q_0, q_6\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$
$(q_6, a) = q_6$ $(q_6, b) = q_6$

not in the same set.

Check for $\{q_4, q_6\}$.

$(q_4, a) = q_6$ $(q_4, b) = q_4$
$(q_6, a) = q_6$ $(q_6, b) = q_6$

In the same set.

Check for $\{q_3, q_5\}$

$(q_3, a) = q_3$ $(q_3, b) = q_5$
$(q_5, a) = q_3$ $(q_5, b) = q_5$

Same set

2's equivalent –

$\{q_0, q_2\}$. $\{q_4, q_6\}$, $\{q_1\}$ $\{q_3, q_5\}$

NANDINI N

3's Equivalent -

check $\{q_0, q_2\}$

$(q_0, a) = q_1$ $(q_0, b) = q_2$

$(q_2, a) = q_1$ $(q_2, b) = q_2$

same set.

check $\{q_4, q_6\}$

$(q_4, a) = q_6$ $(q_4, b) = q_4$

$(q_6, a) = q_6$ $(q_6, b) = q_6$

Same set.

Check $\{q_3, q_5\}$

$(q_3, a) = q_3$ $(q_3, b) = q_5$

$(q_5, a) = q_3$ $(q_5, b) = q_5$

same set

3's equivalent

$\{q_0, q_2\}$ $\{q_4, q_6\}$ $\{q_1\}$ $\{q_3, q_5\}$

These are the final state because we got the same value of 2's and 3's equivalent.